



Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai



U23IT453 – CRYPTOGRAPHY AND CYBER SECURITY LABORATORY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**SRI ESHWAR COLLEGE OF ENGINEERING
KINATHUKADAVU COIMBATORE - 641202**



Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONA FIDE CERTIFICATE

Certified that this is the bonafide record of work done by

Name: Mr. /Ms.

Register No: of 3rd Year

B.E – Computer Science and Engineering in the U23IT453 – CRYPTOGRAPHY AND CYBER

SECURITY LABORATORY during the 6th Semester of the academic year 2025 – 2026 (Even Semester).

Signature of faculty In-charge

Head of the department

Submitted for the practical examinations of Anna University, held on

Internal Examiner

LIST OF EXPERIMENTS

EX. NO	DATE	NAME OF THE EXPERIMENTS	PAGE NO	MARKS	SIGNATURE
1.		Write a program to implement the following cipher techniques to perform encryption and decryption. (a) Caesar Cipher (b) Playfair Cipher (c) Hill Cipher	3, 5, 12		
2.		Write a program to implement the following transposition techniques (a) Rail fence technique –Row major transformation (b) Rail fence technique - Column major transformation	16, 19		
3.		Write a program to implement DES, AES and RSA Encryption algorithm.	24, 26, 28		
4.		Write a program to implement the Diffie-Hellman Key Exchange mechanism. Consider one of the parties as Alice and the other party as bob.	31		
5.		Write a program to calculate the message digest of a text using the SHA-1 algorithm.	34		
6.		Write a program to calculate the message digest of a text using the MD-5 algorithm.	36		
7.		To configure virtual networks using network simulator.	38		
8.		To install and exploit security tools for protecting a network.	40		
9.		To perform the web penetration testing using Burp suite.	42		
10.		Exploit SQL injection to bypass authentication and retrieve sensitive data.	45		

Ex. No: 1 (a)	CAESAR CIPHER
Date:	

Objective:

To implement the Caesar Cipher technique for encryption and decryption.

Principle:

Caesar Cipher is a substitution cipher where each letter in the plaintext is shifted by a fixed number of positions in the alphabet. It is one of the simplest and oldest encryption techniques.

Algorithm:

1. Take input plaintext and shift value.
2. Iterate through each character in the text.
3. Shift letters by the given value while preserving case.
4. Keep non-alphabetic characters unchanged.
5. For decryption, use shift = 26 - original shift.

Implementation:

```
import java.util.Scanner;

public class Main {

    // Method to encrypt the text using Caesar Cipher
    public static String encrypt(String text, int shift) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                result.append((char) ((ch - base + shift) % 26 + base));
            } else {
                result.append(ch);
            }
        }
        return result.toString();
    }

    // Method to decrypt the text using Caesar Cipher
```

```
public static String decrypt(String text, int shift) {  
    return encrypt(text, 26 - shift);  
}  
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Enter text: ");  
    String text = scanner.nextLine();  
    System.out.print("Enter shift value: ");  
    int shift = scanner.nextInt();  
    String encryptedText = encrypt(text, shift);  
    System.out.println("Encrypted Text: " + encryptedText);  
    String decryptedText = decrypt(encryptedText, shift);  
    System.out.println("Decrypted Text: " + decryptedText);  
    scanner.close();  
}  
}
```

Output:

A screenshot of a Java IDE console window. The window has a title bar with standard OS icons. The console output is as follows:
Enter text: HELLO
Enter shift value: 4
Encrypted Text: LIPPS
Decrypted Text: HELLO

Result:

The Caesar Cipher technique for encryption and decryption successfully implemented.

Ex. No: 1 (b)	PLAYFAIR CIPHER
Date:	

Objective:

To implement the Playfair Cipher technique for encryption and decryption.

Principle:

Playfair Cipher is a digraph substitution cipher that encrypts pairs of letters based on a 5×5 matrix generated using a keyword. It enhances security over monoalphabetic ciphers.

Algorithm:

1. Replace 'J' with 'I' and create a 5x5 key matrix.
2. Remove duplicate letters from the key and fill with remaining alphabets.
3. Format plaintext into letter pairs, inserting 'X' if needed.
4. If in same row, replace with next right letter.
5. If in same column, replace with next below letter.
6. Otherwise, swap column positions within a rectangle.
7. Reverse the process for decryption and remove 'X' fillers.

Implementation:

```
package playfairCipher;
import java.util.*;
public class PlayfairCipher {
    public static void main(String []args){
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter the plain text: ");
        String plain = sc.next();
        System.out.println("Enter the key: ");
        String key = sc.next();
        System.out.println("\nEncryption using playfair:\n");
        String pairplain = pair(plain);
        char[][] keymatrix=formKeyMatrix(key);
        System.out.println("Filler letter: X\n");
        System.out.println("5x5 key matrix(considering I and J as a single letter): ");
```

```

for(int i=0;i<5;i++)
{
for(int j=0;j<5;j++)
{
System.out.print(keymatrix[i][j]+" ");
}
System.out.println("");
}
String ciphertext=toCipherOrPlain(keymatrix,pairplain);
System.out.println("\nCorresponding Cipher text: "+ciphertext);
System.out.println("\nDecryption using playfair:\n");
String paircipher=pair(ciphertext);
String pt=toCipherOrPlain(keymatrix,paircipher);
String plaintext=removeFiller(pt);
System.out.println("Cipher text: "+ciphertext);
System.out.println("Corresponding Plain text: "+plaintext);
}
public static String pair(String plain)
{
String pair="";
char filler='X';
for(int i=0;i<plain.length();i+=2)
{
>pair them
if(plain.charAt(i)!=plain.charAt(i+1)) //if the letters to be paired are not same-
{
pair+=plain.charAt(i)+" "+plain.charAt(i+1);
}
else //if the letters to be paired are same->pair the first letter with filler
{

```

```

pair+=plain.charAt(i)+""+filler;
i-=1;
}
}
return pair;
}
public static char[][] formKeyMatrix(String key)
{
key=key.replaceAll("J","I"); //replace all J's with I's in key
char[][] keymatrix=new char[5][5];
int a=0;
int Jascii=74;
Set<Character> unique= new LinkedHashSet<Character>();
String uniquestr="";
for(int p=0;p<key.length();p++)
{
the key
unique.add(key.charAt(p)); //resultant set will contain only unique characters in
}
for(char i=65;i<91;i++)
{
single letter(I))
}
if(i!=Jascii) //The key matrix will not contain J (as I and J are considered as a
unique.add(i);
Iterator<Character> iter=unique.iterator();
while(iter.hasNext())
{
uniquestr+=iter.next();
}
}

```

```

for(int i=0;i<5;i++)
{
for(int j=0;j<5;j++)
{
keymatrix[i][j]=uniquestr.charAt(a);
a++;
}
}
return keymatrix;
}

public static String toCipherOrPlain(char[][] key, String pair)
{
boolean found1=false;
boolean found2=false;
int frow = 0,fcol=0,srow=0,scol=0;
String cipherorplain="";
for(int ind=0;ind+1<pair.length();ind+=2)
{
found1=false;
found2=false;
for(int i=0;i<5;i++)
{
for(int j=0;j<5;j++)
{
if(pair.charAt(ind)==key[i][j])
{
frow=i;
fcol=j;
found1=true;
}
}
}
}

```

```

if(pair.charAt(ind+1)==key[i][j])
{
srow=i;
scol=j;
found2=true;
}
if(found1 & found2)
break;
}
if(found1 & found2)
break;
}
if(frow==srow) //case I - both the letters are in the same row
{
if(fcol+1<5)
{
}
else
{
}
cipherorplain+=key[frow][fcol+1];
cipherorplain+=key[frow][0];
if(scol+1<5)
{
cipherorplain+=key[srow][scol+1];
}
else
{
}
}
}

```

```

cipherorplain+=key[frow][0];
else if(fcol==scol) //case II - both the letters are in the same column
{
if(frow+1<5)
{
}
else
{
}
cipherorplain+=key[frow+1][fcol];
cipherorplain+=key[0][fcol];
if(srow+1<5)
{
}
else
{
}
}
cipherorplain+=key[srow+1][fcol];
cipherorplain+=key[0][fcol];
column
else // Case III - both the letters are neither in the same row nor in the same
{
cipherorplain+=key[frow][scol];
cipherorplain+=key[srow][fcol];
}
}
return cipherorplain;
}
public static String removeFiller(String text)

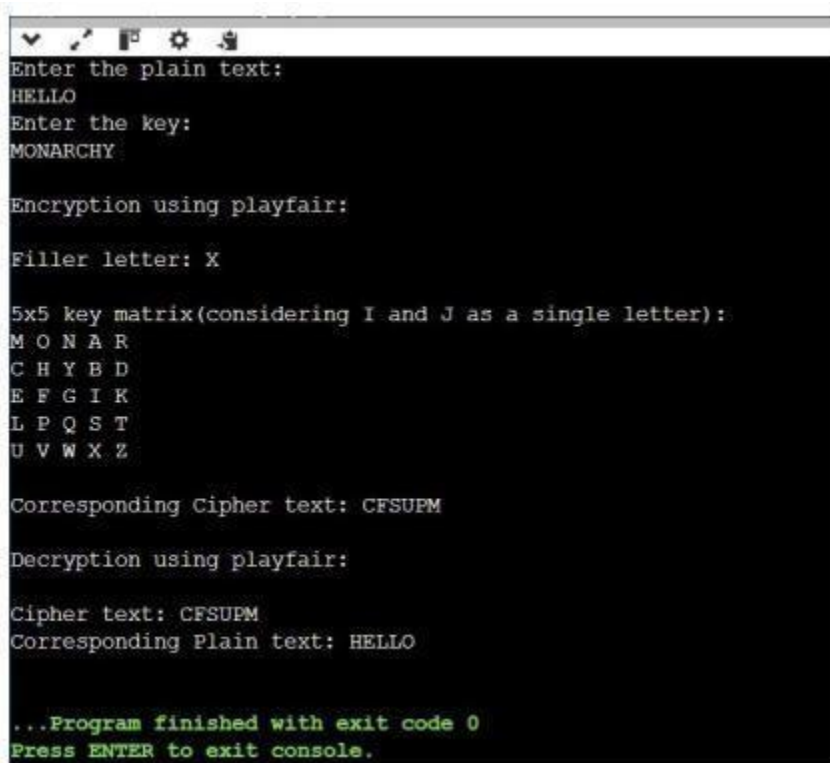
```

```

{
StringBuilder ct=new StringBuilder(text);
for(int i=0;i<ct.length();i++)
{
if(ct.charAt(i)=='X')
{
ct.deleteCharAt(i);
}
}
return ct.toString();
}

```

Output:



```

Enter the plain text:
HELLO
Enter the key:
MONARCHY

Encryption using playfair:

Filler letter: X

5x5 key matrix(considering I and J as a single letter):
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Corresponding Cipher text: CFSUPM

Decryption using playfair:

Cipher text: CFSUPM
Corresponding Plain text: HELLO

...Program finished with exit code 0
Press ENTER to exit console.

```

Result:

The Playfair Cipher technique for encryption and decryption successfully implemented.

Ex. No: 1 (c)	HILL CIPHER
Date:	

Objective:

To implement the Hill Cipher technique for encryption and decryption.

Principle:

Hill Cipher is a polygraphic substitution cipher that uses matrix multiplication for encryption. It operates on block sizes (2x2 or 3x3) and requires matrix inversion for decryption.

Algorithm:

1. Prepare Key Matrix: Define a 3×3 key matrix and its inverse.
2. Convert Plaintext: Convert characters to numerical positions (A=0, B=1, ..., Z=25).
3. Padding: If the plaintext length is not a multiple of 3, add padding ('X').
4. Encryption: Multiply each 3-letter block by the key matrix and take modulo 26.
5. Ciphertext Mapping: Convert the resulting numbers back to letters.
6. Decryption: Multiply each 3-letter block of ciphertext by the inverse key matrix and take modulo 26.
7. Retrieve Plaintext: Convert numbers back to letters and remove padding.

Implementation:

```
class hillCipher {
    /* 3x3 key matrix for 3 characters at once */
    public static int[][] keymat = new int[][] { { 1, 2, 1 }, { 2, 3, 2 },
        { 2, 2, 1 } }; /* key inverse matrix */
    public static int[][] invkeymat = new int[][] { { -1, 0, 1 }, { 2, -1, 0 }, { -2, 2, -1 }
    };
    public static String key = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    private static String encode(char a, char b, char c) {
        String ret = "";
        int x, y, z;
        int posa = (int) a - 65;
        int posb = (int) b - 65;
        int posc = (int) c - 65;
```

```

x = posa * keymat[0][0] + posb * keymat[1][0] + posc * keymat[2][0];
y = posa * keymat[0][1] + posb * keymat[1][1] + posc * keymat[2][1];
z = posa * keymat[0][2] + posb * keymat[1][2] + posc * keymat[2][2];
a = key.charAt(x % 26);
b = key.charAt(y % 26);
c = key.charAt(z % 26);
ret = "" + a + b + c;
return ret;
}

private static String decode(char a, char b, char c) {
String ret = "";
int x, y, z;
int posa = (int) a - 65;
int posb = (int) b - 65;
int posc = (int) c - 65;
x = posa * invkeymat[0][0] + posb * invkeymat[1][0] + posc *
invkeymat[2][0];
y = posa * invkeymat[0][1] + posb * invkeymat[1][1] + posc *
invkeymat[2][1];
z = posa * invkeymat[0][2] + posb * invkeymat[1][2] + posc *
invkeymat[2][2];
a = key.charAt((x % 26 < 0) ? (26 + x % 26) : (x % 26));
b = key.charAt((y % 26 < 0) ? (26 + y % 26) : (y % 26));
c = key.charAt((z % 26 < 0) ? (26 + z % 26) : (z % 26));
ret = "" + a + b + c;
return ret;
}

public static void main(String[] args) throws java.lang.Exception {
String msg;
String enc = "";

```

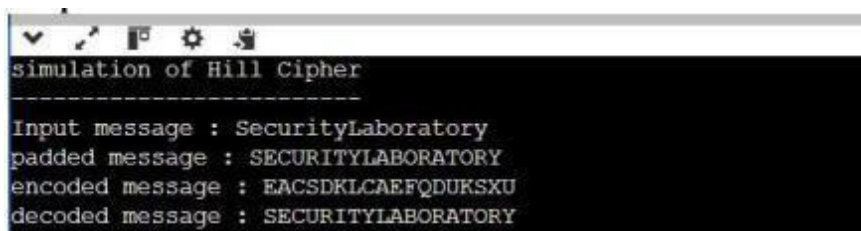
```

String dec = "";
int n;

msg = ("SecurityLaboratory");
System.out.println("simulation of Hill Cipher\n----- ");
System.out.println("Input message : " + msg);
msg = msg.toUpperCase();
msg = msg.replaceAll("\\s", "");
/* remove spaces */ n = msg.length() % 3;
/* append padding text X */ if (n != 0) {
for (int i = 1; i <= (3 - n); i++) {
msg += 'X';
}}
System.out.println("padded message : " + msg);
char[] pdchars = msg.toCharArray();
for (int i = 0; i < msg.length(); i += 3) {
enc += encode(pdchars[i], pdchars[i + 1], pdchars[i + 2]);
}
System.out.println("encoded message : " + enc);
char[] dechars = enc.toCharArray();
for (int i = 0; i < enc.length(); i += 3) {
dec += decode(dechars[i], dechars[i + 1], dechars[i + 2]);
}
System.out.println("decoded message : " + dec);
}
}

```

Output:



```

simulation of Hill Cipher
-----
Input message : SecurityLaboratory
padded message : SECURITYLABORATORY
encoded message : EACSDKLCAEFQDUKSXU
decoded message : SECURITYLABORATORY

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The Hill Cipher technique for encryption and decryption successfully implemented.

Ex. No: 2 (a)	RAIL FENCE TECHNIQUE – ROW MAJOR TRANSFORMATION
Date:	

Objective:

To implement the Rail Fence Technique using a Row-Major Transformation for encrypting and decrypting a message.

Principle:

The Rail Fence cipher is a transposition cipher that rearranges the characters of a plaintext by writing them in a zigzag pattern across multiple rows and then reading them row by row. In the row-major transformation, the text is written in a rail fence pattern and then read row-wise.

Algorithm:

1. Accept the plaintext and depth (number of rows), then arrange the characters in a zigzag pattern.
2. Read the character's row by row to obtain the cipher text.
3. Create an empty matrix and mark the zigzag pattern positions for decryption.
4. Fill the marked positions row-wise with the cipher text characters.
5. Read the matrix in a zigzag pattern to reconstruct the original plaintext.

Implementation:

```

class RailFenceCipher {
public static String encrypt(String text, int depth) {
    char[][] rail = new char[depth][text.length()];
    int row = 0, direction = 1;
    // Fill matrix in zigzag pattern
    for (int i = 0; i < text.length(); i++) {
        rail[row][i] = text.charAt(i);
        row += direction;
        if (row == 0 || row == depth - 1) direction *= -1;
    }
    // Read row-wise to get ciphertext
    StringBuilder cipherText = new StringBuilder();
    for (char[] chars : rail)
        for (char ch : chars)
            if (ch != 0) cipherText.append(ch);
    return cipherText.toString();
}
}

```

```

    }
    public static String decrypt(String cipherText, int depth) {
        char[][] rail = new char[depth][cipherText.length()];
        int row = 0, direction = 1;
        // Mark zigzag pattern positions
        for (int i = 0; i < cipherText.length(); i++) {
            rail[row][i] = '*';
            row += direction;
            if (row == 0 || row == depth - 1) direction *= -1;
        }
        // Fill marked positions with ciphertext characters
        int index = 0;
        for (int i = 0; i < depth; i++)
            for (int j = 0; j < cipherText.length(); j++)
                if (rail[i][j] == '*' && index < cipherText.length())
                    rail[i][j] = cipherText.charAt(index++);
        // Read zigzag pattern to get plaintext
        StringBuilder plainText = new StringBuilder();
        row = 0;
        direction = 1;
        for (int i = 0; i < cipherText.length(); i++) {
            plainText.append(rail[row][i]);
            row += direction;
            if (row == 0 || row == depth - 1) direction *= -1;
        }
        return plainText.toString();
    }

    public static void main(String[] args) {
        String text = "HELLO RAIL FENCE".replace(" ", "");
        int depth = 3;
        String cipherT
        ext = encrypt(text, depth);
    }

```

```
String decryptedText = decrypt(cipherText, depth);  
System.out.println("Plaintext: " + text);  
System.out.println("Ciphertext: " + cipherText);  
System.out.println("Decrypted Text: " + decryptedText);  
}  
}
```

Output:



```
Plaintext: HELLO RAIL FENCE  
Ciphertext: HOLCEL RIFNELAE  
Decrypted Text: HELLO RAIL FENCE
```

Result:

The Rail Fence Technique using a Row-Major Transformation technique for encryption and decryption successfully implemented.

Ex. No: 2 (b)	RAIL FENCE TECHNIQUE – COLUMN MAJOR TRANSFORMATION
Date:	

Objective:

To implement the Rail Fence Technique using a Column-Major Transformation for encrypting and decrypting a message.

Principle:

The Rail Fence cipher is a transposition cipher that rearranges the characters of a plaintext by writing them in a zigzag pattern across multiple rows. In the column-major transformation, after arranging the text in a rail fence pattern, it is read column-wise instead of row-wise.

Algorithm:

1. Accept the plaintext and the depth (number of rows), then arrange the characters in a zigzag manner across the given number of rows.
2. Read the character's column by column to obtain the cipher text.
3. Create an empty matrix and mark the zigzag pattern positions for decryption.
4. Fill the marked positions column-wise with the cipher text characters.
5. Read the matrix in a zigzag pattern to reconstruct the original plaintext.

Implementation:

```
import java.util.Arrays;

class RailFence {
    // function to encrypt a message
    public static String encryptRailFence(String text, int key)
    {
        // create the matrix to cipher plain text
        // key = rows , length(text) = columns
        char[][] rail = new char[key][text.length()];
        // filling the rail matrix to distinguish filled
        // spaces from blank ones
        for (int i = 0; i < key; i++)
            Arrays.fill(rail[i], '\n');
        boolean dirDown = false;
        int row = 0, col = 0;
        for (int i = 0; i < text.length(); i++) {
            // check the direction of flow
```

```

// reverse the direction if we've just
// filled the top or bottom rail
if(row == 0 || row == key - 1)
    dirDown = !dirDown;
// fill the corresponding alphabet
rail[row][col++] = text.charAt(i);
// find the next row using direction flag
if (dirDown)
    row++;
else
    row--;
}
// now we can construct the cipher using the rail
// matrix
StringBuilder result = new StringBuilder();
for (int i = 0; i < key; i++)
    for (int j = 0; j < text.length(); j++)
        if (rail[i][j] != '\n')
            result.append(rail[i][j]);
return result.toString();
}
// This function receives cipher-text and key
// and returns the original text after decryption
public static String decryptRailFence(String cipher,int key)
{
// create the matrix to cipher plain text
// key = rows , length(text) = columns
char[][] rail = new char[key][cipher.length()];
// filling the rail matrix to distinguish filled
// spaces from blank ones
for (int i = 0; i < key; i++)

```

```

Arrays.fill(rail[i], '\n');
// to find the direction
boolean dirDown = true;
int row = 0, col = 0;
// mark the places with '*'
for (int i = 0; i < cipher.length(); i++) {
// check the direction of flow
if (row == 0)
dirDown = true;
if (row == key - 1)
dirDown = false;
// place the marker
rail[row][col++] = '*';
// find the next row using direction flag
if (dirDown)
row++;
else
}
row--;
// now we can construct the fill the rail matrix
int index = 0;
for (int i = 0; i < key; i++)
for (int j = 0; j < cipher.length(); j++)
if (rail[i][j] == '*'
&& index < cipher.length())
rail[i][j] = cipher.charAt(index++);
StringBuilder result = new StringBuilder();
row = 0;
col = 0;
for (int i = 0; i < cipher.length(); i++) {
// check the direction of flow

```

```

if (row == 0)
dirDown = true;
if (row == key - 1)
dirDown = false;

// place the marker
if (rail[row][col] != '*')
result.append(rail[row][col++]);
// find the next row using direction flag
if (dirDown)
row++;
else
}
row--;
return result.toString();
}

// driver program to check the above functions
public static void main(String[] args)
{
// Encryption
System.out.println("Encrypted Message: ");
System.out.println(
encryptRailFence("attack at once", 2));
System.out.println(
encryptRailFence("GeeksforGeeks ", 3));
System.out.println(
encryptRailFence("defend the east wall", 3));
// Now decryption of the same cipher-text
System.out.println("\nDecrypted Message: ");
System.out.println(
decryptRailFence("atc toctaka ne", 2));
System.out.println(

```

```

decryptRailFence("GsGsekfreak eoe", 3));
System.out.println(
decryptRailFence("dnhaweedtees alf tl", 3));
}
}

```

Output:



```

Encrypted Message:
atc toctaka ne
GsGsekfreak eoe
dnhaweedtees alf  tl

Decrypted Message:
attack at once
GeeksforGeeks
delendfthe east wal

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The Rail Fence Technique using a Column-Major Transformation technique for encryption and decryption successfully implemented.

Ex. No: 3 (a)	DATA ENCRYPTION STANDARD
Date:	

Objective:

To implement the Data Encryption Standard (DES) algorithm for secure encryption and decryption of a given message.

Principle:

The Data Encryption Standard (DES) is a symmetric-key block cipher that encrypts data in 64-bit blocks using a 56-bit key. It follows a Feistel network with 16 rounds of processing, including permutations, substitutions, and XOR operations.

Algorithm:

1. Accept a plaintext message and a secret key (both 64-bit).
2. Apply an initial permutation (IP) on the plaintext.
3. Divide the permuted text into two halves (Left and Right).
4. Perform 16 rounds of Feistel function transformations, including expansion, substitution, permutation, and XOR with round keys.
5. Swap the halves after 16 rounds and apply the final permutation (FP).
6. Convert the final binary output to ciphertext.
7. Decryption follows the same process in reverse, using the same key.

Implementation:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

class DESAlgorithm {

    public static SecretKey generateKey() throws Exception {
        KeyGenerator keyGen = KeyGenerator.getInstance("DES");
        keyGen.init(56); // DES key size
        return keyGen.generateKey();
    }

    public static String encrypt(String plainText, SecretKey key) throws Exception {
        Cipher cipher = Cipher.getInstance("DES");
        cipher.init(Cipher.ENCRYPT_MODE, key);
```

```

byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());
return Base64.getEncoder().encodeToString(encryptedBytes);
}

public static String decrypt(String cipherText, SecretKey key) throws Exception {
    Cipher cipher = Cipher.getInstance("DES");
    cipher.init(Cipher.DECRYPT_MODE, key);
    byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(cipherText));
    return new String(decryptedBytes);
}

public static void main(String[] args) {
    try {
        SecretKey key = generateKey();
        String text = "HELLODES";
        String encryptedText = encrypt(text, key);
        String decryptedText = decrypt(encryptedText, key);
        System.out.println("Plaintext: " + text);
        System.out.println("Ciphertext: " + encryptedText);
        System.out.println("Decrypted Text: " + decryptedText);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Output:



```

Plaintext: HELLODES
Ciphertext: FpWzEmnsvbdNOxvzkhsuQ==
Decrypted Text: HELLODES

```

Result:

The Data Encryption Standard (DES) algorithm technique for encryption and decryption successfully implemented.

Ex. No: 3 (b)	ADVANCED ENCRYPTION STANDARD
Date:	

Objective:

To implement the Advanced Encryption Standard (AES) algorithm for secure encryption and decryption of a given message.

Principle:

The Advanced Encryption Standard (AES) is a symmetric-key block cipher that encrypts data in 128-bit blocks using a 128-bit, 192-bit, or 256-bit key. It operates in multiple rounds, including substitution,

permutation, mixing, and XOR operations with round keys.

Algorithm:

1. Accept a plaintext message and a secret key (128, 192, or 256 bits).
2. Convert plaintext into a byte array and apply an initial transformation.
3. Perform multiple encryption rounds (10 for 128-bit keys, 12 for 192-bit, and 14 for 256-bit).
4. Each round includes SubBytes (S-Box substitution), ShiftRows, MixColumns, and AddRoundKey transformations.
5. The final round omits the MixColumns step.
6. Convert the final encrypted data into ciphertext.
7. Decryption follows the same process in reverse, using the same key.

Implementation:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

class AESAlgorithm {
    public static SecretKey generateKey() throws Exception {
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");
        keyGen.init(128); // AES key size (128, 192, or 256 bits)
        return keyGen.generateKey();
    }
}

public static String encrypt(String plainText, SecretKey key) throws Exception {
    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.ENCRYPT_MODE, key);
```

```

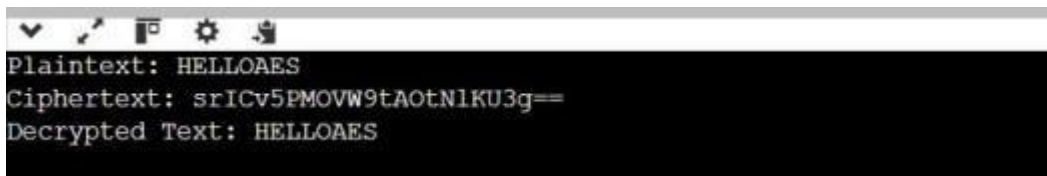
byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());
return Base64.getEncoder().encodeToString(encryptedBytes);
}

public static String decrypt(String cipherText, SecretKey key) throws Exception {
    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, key);
    byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(cipherText));
    return new String(decryptedBytes);
}

public static void main(String[] args) {
    try {
        SecretKey key = generateKey();
        String text = "HELLOAES";
        String encryptedText = encrypt(text, key);
        String decryptedText = decrypt(encryptedText, key);
        System.out.println("Plaintext: " + text);
        System.out.println("Ciphertext: " + encryptedText);
        System.out.println("Decrypted Text: " + decryptedText);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Output:



```

Plaintext: HELLOAES
Ciphertext: srICv5PMOVW9tAotNlKU3g==
Decrypted Text: HELLOAES

```

Result:

The Advanced Encryption Standard (AES) algorithm technique for encryption and decryption successfully implemented.

Objective:

To implement the RSA Encryption Algorithm for secure data encryption and decryption.

Principle:

RSA (Rivest-Shamir-Adleman) is an asymmetric cryptographic algorithm that uses two keys: a public key for encryption and a private key for decryption. It is based on the mathematical difficulty of factoring large prime numbers.

Algorithm:

1. Select two large prime numbers, p and q .
2. Compute $n = p \times q$ (modulus) and $\phi(n) = (p - 1) \times (q - 1)$ (Euler's totient function).
3. Choose an encryption key e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
4. Compute the decryption key d such that $d \equiv e^{-1} \pmod{\phi(n)}$ (modular inverse of e).
5. Encrypt plaintext M using the public key: $C = M^e \pmod{n}$.
6. Decrypt ciphertext C using the private key: $M = C^d \pmod{n}$.

Implementation:

```
import java.math.BigInteger;
import java.security.SecureRandom;

class RSAAlgorithm {
    private BigInteger n, d, e;
    private int bitlen = 1024;

    public RSAAlgorithm() {
        SecureRandom random = new SecureRandom();
        BigInteger p = BigInteger.probablePrime(bitlen / 2, random);
        BigInteger q = BigInteger.probablePrime(bitlen / 2, random);
        n = p.multiply(q);
        BigInteger phi = (p.subtract(BigInteger.ONE)).multiply(q.subtract(BigInteger.ONE));
        e = BigInteger.probablePrime(bitlen / 2, random);
        while (phi.gcd(e).compareTo(BigInteger.ONE) > 0 && e.compareTo(phi) < 0) {
            e = e.add(BigInteger.ONE);
        }
        d = e.modInverse(phi);
    }
}
```

```

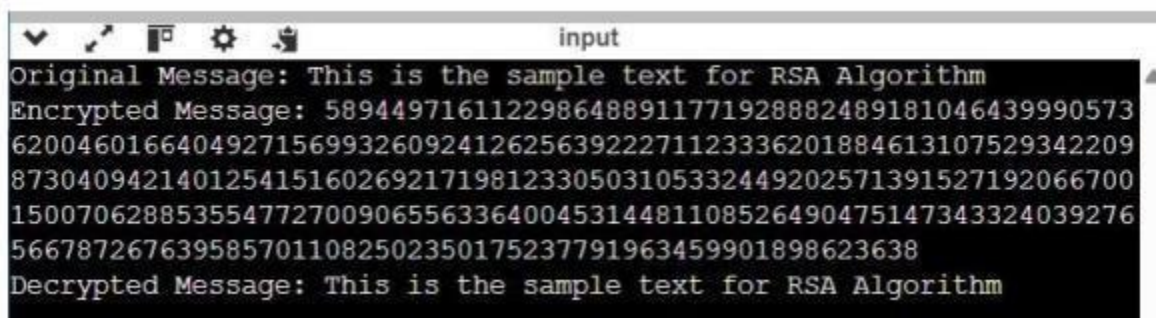
    }
    public BigInteger encrypt(BigInteger message) {
        return message.modPow(e, n);
    }

    public BigInteger decrypt(BigInteger cipher) {
        return cipher.modPow(d, n);
    }

    public static void main(String[] args) {
        RSAAlgorithm rsa = new RSAAlgorithm();
        String message = "This is the sample text for RSA Algorithm";
        BigInteger messageInt = new BigInteger(message.getBytes());
        BigInteger cipherText = rsa.encrypt(messageInt);
        BigInteger decryptedMessage = rsa.decrypt(cipherText);
        System.out.println("Original Message: " + message);
        System.out.println("Encrypted Message: " + cipherText);
        System.out.println("Decrypted Message: " + new String(decryptedMessage.toByteArray()));
    }
}

```

Output:



```

input
Original Message: This is the sample text for RSA Algorithm
Encrypted Message: 5894497161122986488911771928882489181046439990573
62004601664049271569932609241262563922271123336201884613107529342209
87304094214012541516026921719812330503105332449202571391527192066700
15007062885355477270090655633640045314481108526490475147343324039276
5667872676395857011082502350175237791963459901898623638
Decrypted Message: This is the sample text for RSA Algorithm

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The RSA Encryption and Decryption process was successfully implemented, demonstrating secure communication using asymmetric cryptography.

Ex. No: 4	DIFFIE – HELLMAN KEY EXCHANGE MECHANISM
Date:	

Objective:

To implement the Diffie-Hellman Key Exchange Mechanism between two parties: Alice and Bob, to establish a shared secret key over an insecure channel.

Principle:

The Diffie-Hellman Key Exchange is a cryptographic method that allows two parties to establish a shared secret key without directly transmitting it. It is based on modular exponentiation and the difficulty of the discrete logarithm problem.

Algorithm:

1. Alice and Bob agree on a large prime number p and a base g (both public values).
2. Alice selects a private key a and computes $A = g^a \text{ mod } p$, then sends A to Bob.
3. Bob selects a private key b and computes $B = g^b \text{ mod } p$, then sends B to Alice.
4. Alice computes the shared key: $S = B^a \text{ mod } p$.
5. Bob computes the shared key: $S = A^b \text{ mod } p$.
6. Since S is the same for both, it becomes the shared secret key.

Implementation:

```
import java.math.BigInteger;
import java.security.SecureRandom;

class DiffieHellman {
    private static final int BIT_LENGTH = 512;
    private static final SecureRandom random = new SecureRandom();

    public static void main(String[] args) {
        // Publicly known prime (p) and base (g)
        BigInteger p = BigInteger.probablePrime(BIT_LENGTH, random);
        BigInteger g = BigInteger.probablePrime(BIT_LENGTH / 2, random);

        // Alice's private key (a) and public value (A)
        BigInteger a = new BigInteger(BIT_LENGTH, random);
        BigInteger A = g.modPow(a, p);

        // Bob's private key (b) and public value (B)
        BigInteger b = new BigInteger(BIT_LENGTH, random);
```

```

BigInteger B = g.modPow(b, p);

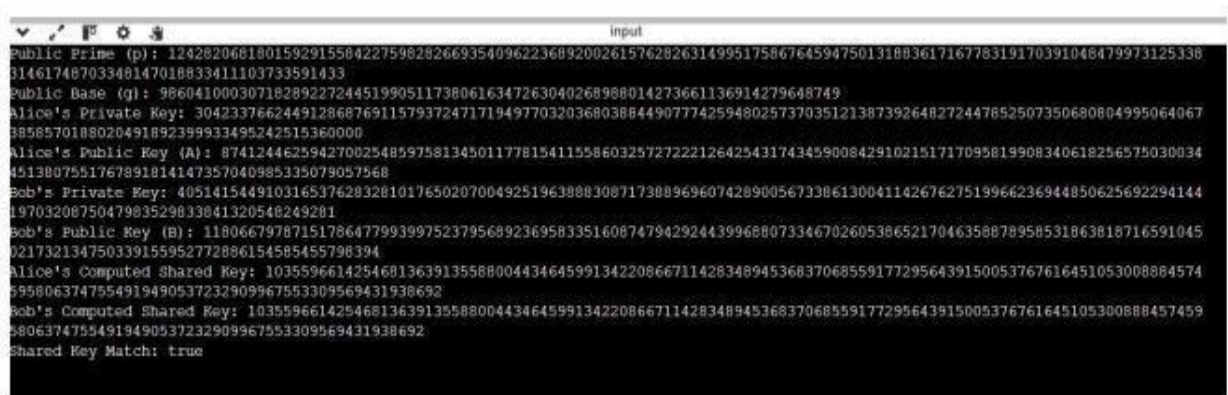
// Compute shared secret keys
BigInteger aliceSharedKey = B.modPow(a, p);
BigInteger bobSharedKey = A.modPow(b, p);

// Display results
System.out.println("Public Prime (p): " + p);
System.out.println("Public Base (g): " + g);
System.out.println("Alice's Private Key: " + a);
System.out.println("Alice's Public Key (A): " + A);
System.out.println("Bob's Private Key: " + b);
System.out.println("Bob's Public Key (B): " + B);
System.out.println("Alice's Computed Shared Key: " + aliceSharedKey);
System.out.println("Bob's Computed Shared Key: " + bobSharedKey);

// Verify if both computed keys are equal
System.out.println("Shared Key Match: " + aliceSharedKey.equals(bobSharedKey));
}
}

```

Output:



```

Public Prime (p): 124282068180159291558422759828266935409622368920026157628263149951758676459475013188361716778319170391046479973125338
31461748703348147018833411103733591433
Public Base (g): 98604100030718289227244519905117380616347263040268988014273661136914279648749
Alice's Private Key: 304233766244912868769115793724717194977032036803884490777425948025737035121387392648272447852507350680804995064067
3858570188020491892399933495242515360000
Alice's Public Key (A): 874124462594270025485975813450117781541155860325727222126425431743459008429102151717095819908340618256575030034
4513807551767891814147357040985335079057568
Bob's Private Key: 40514154491031653762832810176502070049251963888308717388969607428900567338613004114267627519966236944850625692294144
1970320875047983529833841320548249281
Bob's Public Key (B): 11806679787151786477993997523795689236958335160874794292443996880733467026053865217046358878958531863818716591045
021732134750339155952772886154585455798394
Alice's Computed Shared Key: 1035596614254681363913558800443464599134220866711428348945368370685591772956439150053767616451053008884574
5958063747554919490537232909967553309569431938692
Bob's Computed Shared Key: 103559661425468136391355880044346459913422086671142834894536837068559177295643915005376761645105300888457459
58063747554919490537232909967553309569431938692
Shared Key Match: true

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The Diffie-Hellman Key Exchange Mechanism was successfully implemented, enabling Alice and Bob to securely establish a shared secret key over an insecure channel.

Ex. No: 5	SHA – 1 ALGORITHM
Date:	

Objective:

To implement a program that calculates the message digest of a text using the SHA-1 Algorithm.

Principle:

SHA-1 (Secure Hash Algorithm 1) is a cryptographic hash function that takes an input and produces a fixed-size 160-bit (20-byte) hash value. It is commonly used for verifying data integrity.

Algorithm:

1. Accept input text from the user.
2. Convert the input text into a byte array.
3. Use the SHA-1 algorithm to compute the hash.
4. Convert the hash into a readable hexadecimal format.
5. Display the generated message digest.

Implementation:

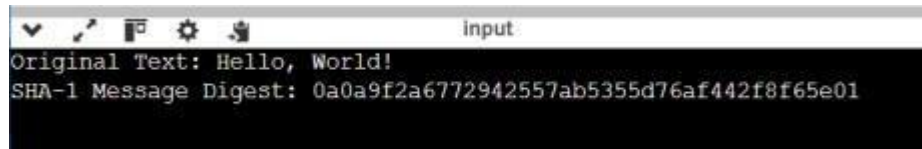
```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
class SHA1Hashing {
public static String generateSHA1(String input) {
try {
// Create MessageDigest instance for SHA-1
MessageDigest md = MessageDigest.getInstance("SHA-1");
// Compute the hash
byte[] messageDigest = md.digest(input.getBytes());
// Convert byte array to hexadecimal format
StringBuilder hexString = new StringBuilder();
for (byte b : messageDigest) {
hexString.append(String.format("%02x", b));
}
return hexString.toString();
} catch (NoSuchAlgorithmException e) {
throw new RuntimeException(e);
}
}
```

```

public static void main(String[] args) {
    String text = "Hello, World!";
    String sha1Hash = generateSHA1(text);
    System.out.println("Original Text: " + text);
    System.out.println("SHA-1 Message Digest: " + sha1Hash);}}

```

Output:



The screenshot shows a terminal window titled 'input' with the following output:

```

Original Text: Hello, World!
SHA-1 Message Digest: 0a0a9f2a6772942557ab5355d76af442f8f65e01

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The program successfully calculates the SHA-1 message digest of a given text, ensuring data integrity verification.

Ex. No: 6	MD5 ALGORITHM
Date:	

Objective:

To implement a program that calculates the message digest of a text using the MD5 Algorithm.

Principle:

MD5 (Message-Digest Algorithm 5) is a cryptographic hash function that takes an input and produces a fixed-size 128-bit (16-byte) hash value. It is commonly used for verifying data integrity.

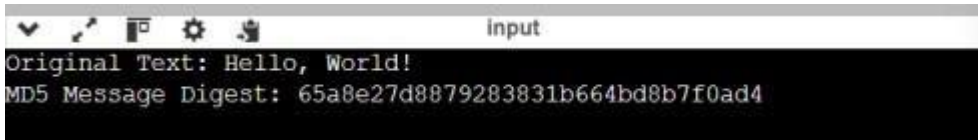
Algorithm:

1. Accept input text from the user.
2. Convert the input text into a byte array.
3. Use the MD5 algorithm to compute the hash.
4. Convert the hash into a readable hexadecimal format.
5. Display the generated message digest.

Implementation:

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
class MD5Hashing {
public static String generateMD5(String input) {
try {
// Create MessageDigest instance for MD5
MessageDigest md = MessageDigest.getInstance("MD5");
// Compute the hash
byte[] messageDigest = md.digest(input.getBytes());
// Convert byte array to hexadecimal format
StringBuilder hexString = new StringBuilder();
for (byte b : messageDigest) {
hexString.append(String.format("%02x", b));
}
return hexString.toString();
} catch (NoSuchAlgorithmException e) {
throw new RuntimeException(e);
}
}
```

```
public static void main(String[] args) {  
    String text = "Hello, World!";  
    String md5Hash = generateMD5(text);  
    System.out.println("Original Text: " + text);  
    System.out.println("MD5 Message Digest: " + md5Hash);}}
```

Output:A screenshot of a Java IDE's output window. The window has a title bar with standard icons and the text 'input'. The output text is displayed on a black background with white text. It shows 'Original Text: Hello, World!' on the first line and 'MD5 Message Digest: 65a8e27d8879283831b664bd8b7f0ad4' on the second line.

```
Original Text: Hello, World!  
MD5 Message Digest: 65a8e27d8879283831b664bd8b7f0ad4
```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

The program successfully calculates the MD5 message digest of a given text, ensuring data integrity verification.

Ex. No: 7	Configuring Virtual Networks Using Cisco Packet Tracer
Date:	

Objective:

To configure virtual networks using Cisco Packet Tracer and simulate communication between multiple devices across different networks.

Software Required:

Cisco Packet Tracer

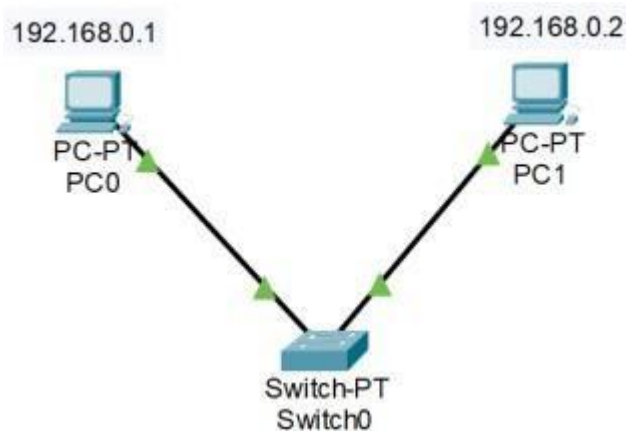
Theory:

A virtual network is a simulated environment that replicates physical network components. It allows testing, learning, and verifying network configurations without needing real hardware. Cisco Packet Tracer is a network simulation tool developed by Cisco that allows users to Create and connect routers, switches, and end devices. Configure IP addresses, routing, and subnetting. Simulate traffic and troubleshoot issues in a virtual environment.

Procedure:

- Step 1: Open Cisco Packet Tracer and create a new project
- Step 2: Drag and drop devices like routers, switches, and PCs into the workspace.
- Step 3: Connect the devices using appropriate cables and assign IPs.
- Step 4: Verify connectivity using the ping command
- Step 5: Test the connectivity and check ping from the command prompt
- Step 6: Save the project and analyze the network behavior.

Implementation:



Output:

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.0.2

Pinging 192.168.0.2 with 32 bytes of data:

Reply from 192.168.0.2: bytes=32 time<1ms TTL=128
Reply from 192.168.0.2: bytes=32 time<1ms TTL=128
Reply from 192.168.0.2: bytes=32 time<1ms TTL=128
Reply from 192.168.0.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

RESULT:

Successfully configured virtual networks using Cisco Packet Tracer and verified communication between devices in different subnets.

Objective:

To install and Exploit Security Tools for Network Protection

Software Required:

Nessus Tool

Theory:

Nessus is an automated penetration testing tool that detect open ports, services, and known vulnerabilities on a target system. It simplifies the reconnaissance and exploitation phases of ethical hacking by combining scanning and exploitation tools under a unified Python interface.

Procedure:

- Step 1: Install Nessus tool
- Step 2: Use the argument to scan the target IP address.
- Step 3: Review generated reports and scanned results
- Step 4: Identify Exploitable Vulnerabilities
- Step 5: Export or Open Reports
- Step 6: Exploit with External Tools
- Step 7: Analyze & Validate the network

Implementation:

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

Result:

Successfully performed an automated vulnerability scan and exploitation on the target system using Nessus tool.

Ex. No: 9	WEB PENETRATION TESTING USING BURP SUITE
Date:	

Objective:

Web Penetration Testing using Burp Suite

Software Required:

Burp Suite tool

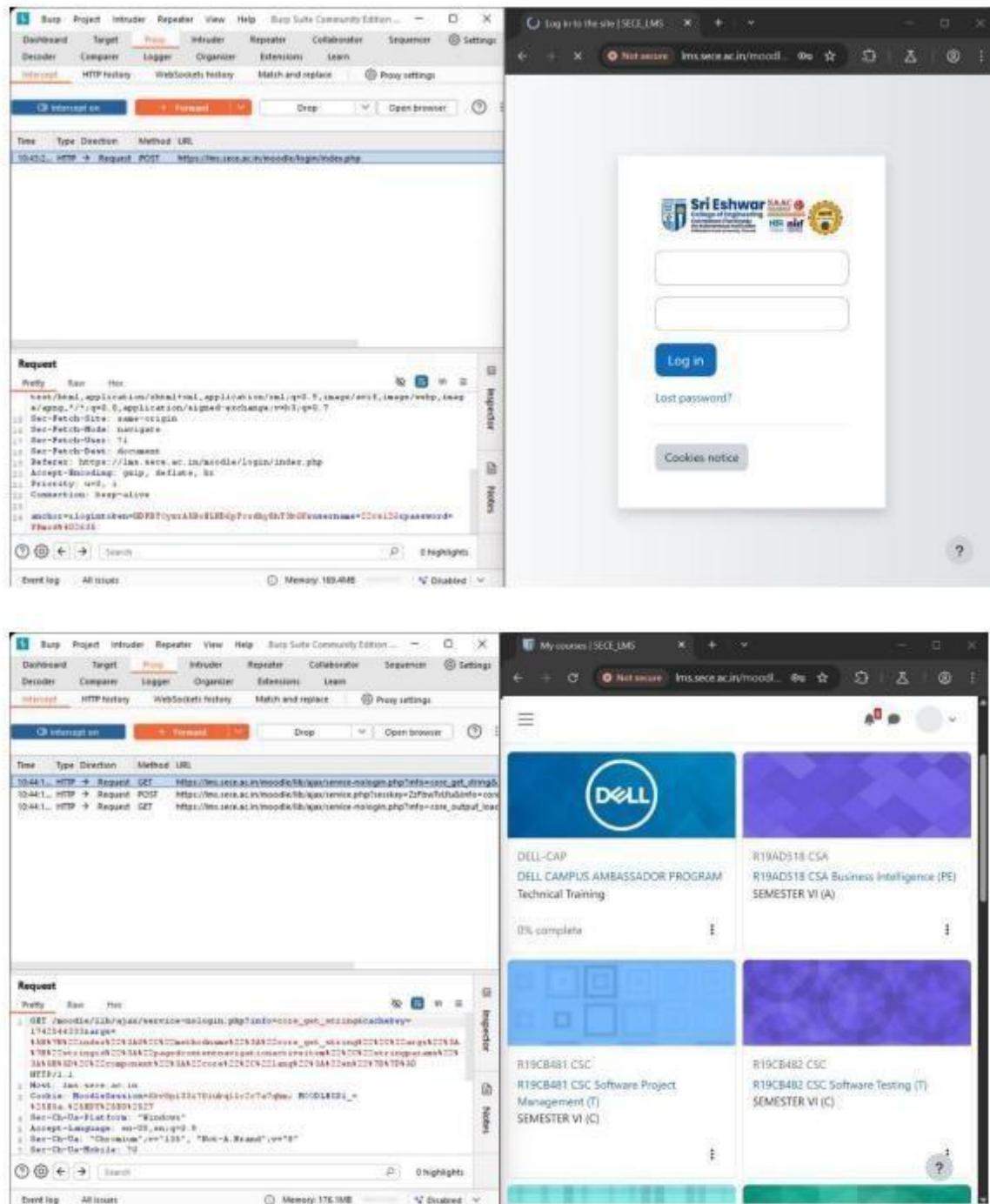
Theory:

Burp Suite is a comprehensive and widely used platform for performing security testing of web applications. Developed by PortSwigger, it acts as an intercepting proxy server, sitting between a user's browser and the target web application. This allows Burp Suite to capture, inspect, and modify HTTP and HTTPS traffic in real time, making it an essential tool for both manual and automated security assessments.

Procedure:

- Step 1: Install and Launch Burp Suite
- Step 2: Intercept and Capture Web Traffic
- Step 3: Review and Analyze Requests
- Step 4: Identify Vulnerabilities
- Step 5: Modify and Replay Requests
- Step 6: Exploit Discovered Vulnerabilities
- Step 7: Analyze and Validate Results
- Step 8: Document Findings and Generate Report

Implementation:



Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

RESULT:

Successfully performed web penetration testing using Burp Suite by intercepting, modifying, and analyzing HTTP requests and responses.

Ex. No: 10	WEB EXPLOIT SQL INJECTION TO BYPASS AUTHENTICATION AND RETRIEVE SENSITIVE DATA
Date:	

Objective:

To understand and demonstrate how SQL Injection vulnerabilities can be exploited to bypass authentication mechanisms and retrieve sensitive data from a vulnerable web application.

Software / Tools Required:

- Vulnerable Web Application (DVWA / Test PHP–MySQL application)
- Web Browser (Chrome / Firefox)
- Local Server Stack (XAMPP / WAMP / LAMP)
- Database: MySQL
- Optional Tool: Burp Suite (for request interception)

Theory:

SQL Injection is one of the most critical web application vulnerabilities, arising when user-supplied input is directly concatenated into SQL queries without proper validation or sanitization. Attackers can manipulate SQL queries by injecting malicious SQL statements, allowing them to bypass login authentication, access unauthorized data, or even modify database contents.

In authentication-based SQL Injection, the attacker exploits poorly written login queries such as:
`SELECT * FROM users WHERE username='input' AND password='input';`

By injecting specially crafted SQL payloads, attackers can alter the logic of the query so that the condition always evaluates to true, thereby gaining unauthorized access. Once authentication is bypassed, sensitive data such as usernames, passwords, and personal records can be retrieved from the database.

Procedure:

Step 1: Setup the Vulnerable Environment

- Install and configure the local server (XAMPP/WAMP).
- Deploy the vulnerable web application.
- Ensure the database connection is active.

Step 2: Analyze the Login Page

- Identify input fields (username and password).
- Observe how input is processed by the server.

Step 3: Test for SQL Injection Vulnerability

- Enter special characters (e.g., ') in input fields.
- Check for database error messages or abnormal behavior.

Step 4: Bypass Authentication

- Inject SQL payloads that modify the login query logic.
- Attempt to log in without valid credentials.

Step 5: Retrieve Sensitive Data

- Identify injectable parameters.
- Use UNION-based or error-based injection to extract database data.

Step 6: Observe Server Responses

- Analyze successful login without valid credentials.
- Verify extracted sensitive information.

Step 7: Validate the Vulnerability

- Confirm that unauthorized access was gained due to SQL Injection.
- Ensure reproducibility of the attack.

Step 8: Document Findings

- Record vulnerable queries.
- Capture screenshots of successful exploitation.

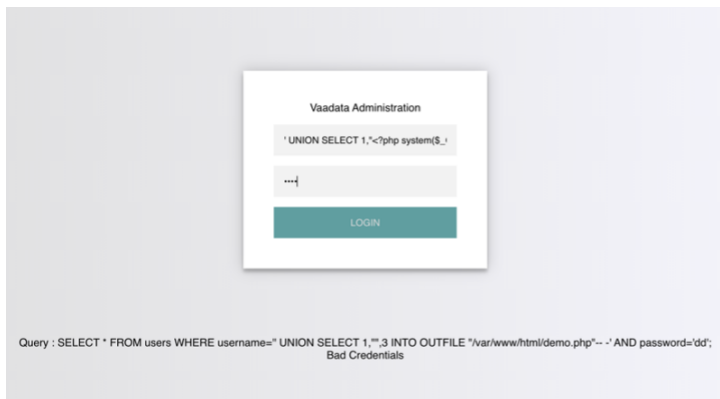
Implementation:

Vulnerable Login Page

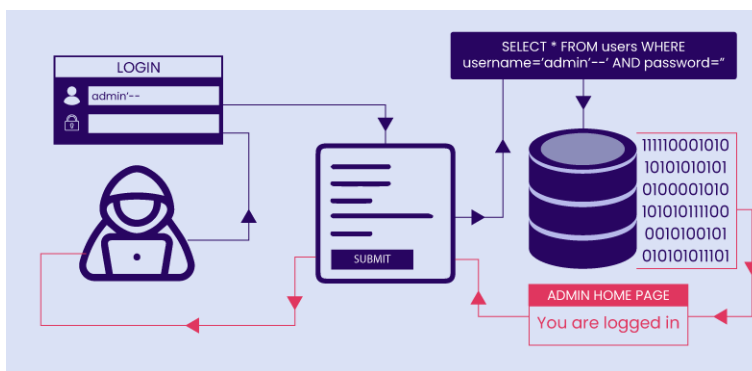


The screenshot shows a web form with a pink header bar containing the text "Please sign-in". Below the header, there are two input fields: "Name" and "Password". The "Name" field is highlighted with a red border. Below the "Password" field, there is a blue "Login" button. At the bottom of the form, there is a link that says "Dont have an account? Please register here".

SQL Injection Payload Entered in Login Fields



Successful Authentication Bypass



Retrieval of Sensitive Database Information

```

MariaDB [demo]> SELECT * FROM users INTO OUTFILE "/tmp/creds.txt";
Query OK, 1 row affected (0.001 sec)

MariaDB [demo]> SELECT LOAD_FILE("/tmp/creds.txt");
+-----+
| LOAD_FILE("/tmp/creds.txt") |
+-----+
| 1  admin  Vaada7aPa55w0rd! |
+-----+
1 row in set (0.000 sec)

```

Criterion	Max	Obtained
Preparation	20	
Implementation	25	
Result	20	
Viva	10	
Total	75	

RESULT:

Successfully exploited an SQL Injection vulnerability to bypass authentication and retrieve sensitive data from a vulnerable web application, demonstrating the risks of improper input validation and insecure query handling